An Internship Report

Of

MERN Full Stack Development Virtual Internship

Submitted

To

School of Computer Science and Engineering
IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of

B.Tech CSE



By

Roll Number of Student:**2410031649**

Name of Student: **SANJIV SINGH**

Batch: 2024-2028

CANDIDATE'S DECLARATION

I, SANJIV SINGH do hereby declare that the internship report titled **"MERN Full Stack Development Virtual Internship"** has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in CSE. All the references have been quoted and I have not taken as such any material from any other source. I affirm to you that this report is my own and has not been submitted to any other institute for any degree/diploma requirement and further I shall be solely responsible for any kind of copyright violation in this regard.

Signature :-
Student Name : SANJIV SINGH
Roll No. : 2410031649

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout the Internship Program. I am thankful for their aspiring guidance, invaluably constructive criticism and friendly advice during this work. I am sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to this work.

I sincerely thank **AICTE and EduSkills** for providing the opportunity to undertake this virtual internship. I am grateful to my industry mentor and the Department of Computer Science for their guidance, support, and encouragement throughout the internship.

I express my gratitude to my parents for their blessings.

Signature
Student Name: SANJIV SINGH
Roll No. :2410031649

TABLE OF CONTENTS

1. Candidate's Declaratio.....i

2. Acknowledgement.....ii

3. Internship Completion Certificate..... -

4. Project Description..... 1

4.1 Introduction 1

4.2 Organization Profile …..... 2

4.3 Internship Overview 3

4.4 Project Objectives 3

4.5 Scope of the Project 4

4.6 Technologies and Tools Used..... 4

4.7 System Architecture 5

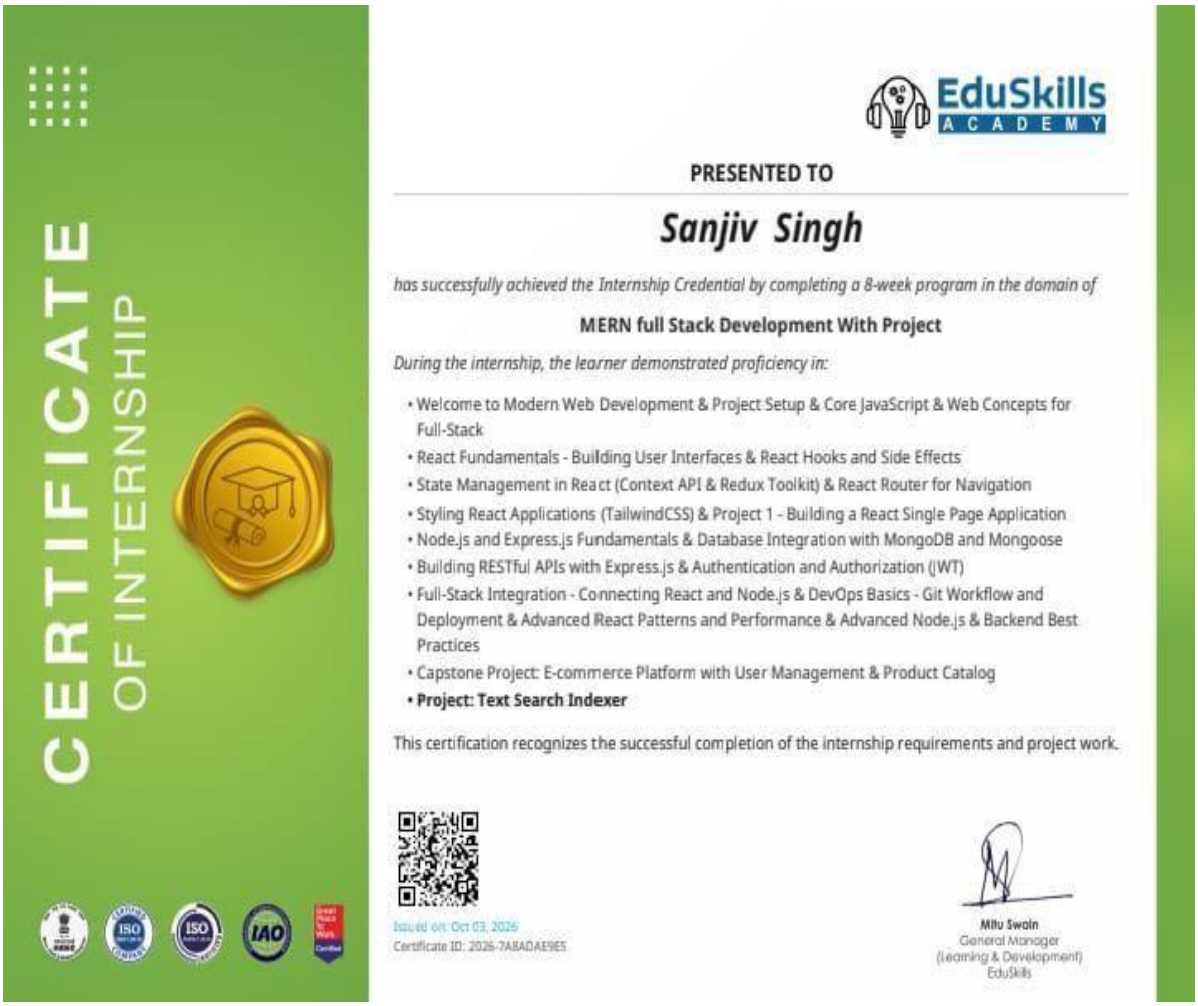
4.8 Methodology 6

4.9 Expected Outcomes 7

4.10 Certificates of Completion and Communication Proof…..... 8

5. Bibliography/References..... 9

3.INTERNSHIP COMPLETION CERTIFICATION:



Internship Offer Letter

EduSkills
Nation Building Through Skills

Ref No: C18Y2026M101533528

Internship Offer Letter

Student Name	: Sanjiv Singh
Stream	: B.Tech
Branch	: B.Tech(Computer Engineering)
Institute Name	: IILM University, Greater Noida
Internship Domain	: MERN full Stack Development + Project
Internship Duration	: August 2026 - October 2026
AICTE Student ID	: STU69ebb4588f93c1777054808

We are pleased to inform you that you have been selected for the 8-Week **AICTE – EduSkills Virtual Internship Program**.

During this internship, you will learn and demonstrate industry-relevant skills that will enhance your employability and strengthen your confidence in the subject area.

The program is designed to provide structured learning combined with practical exposure.

Internship Structure & Evaluation:

- The internship will follow a **week-wise structured learning plan**.
- You will be required to complete **weekly assessments** to track your progress and understanding.
- Certain modules will require submission of **project documentation and assignments**.
- In the final week, you must appear for a **Final Assessment Test**.
- The final grade will be based on your overall performance, including weekly assessments, project submissions, and the final test.

Upon successful completion of all required steps and assessments, you will be awarded a **Virtual Internship Certificate**.

Please note that this offer letter will be considered valid only after the successful completion of the internship program and fulfillment of all assessment requirements, including obtaining the final completion certificate. Without the final completion certificate, this offer letter shall not be treated as a valid document for any official purpose.

PROJECT DESCRIPTION

1. Introduction :

Industry-oriented learning through internships plays a crucial role in bridging the gap between academic knowledge and real-world applications. This online virtual internship provided hands-on exposure to **MERN full Stack Development With Project** and its application in modern software solutions.

2. Organization Profile :

Organization Name: EduSkills Academy

Domain: EdTech & Skill Development

Services: Virtual Internships

Vision: To empower students with industry-relevant skills.

3. Internship Overview :

• Internship Mode: **Online / Virtual**

• Duration: **8 Weeks**

• Platform Used: **EduSkills LMS**

• Role: **Intern**

4. Problem Statement & Objectives :

Problem Statement:

The Text Search Indexer is a sophisticated system engineered to process and analyze large volumes of textual data. Its primary function is to create an efficient index that allows for rapid searching across multiple text documents. The system meticulously breaks down text into individual words (tokenization) and standardizes them (normalization) to ensure search accuracy. It then builds an inverted index where each word is associated with the specific lines it appears on. The indexer supports advanced search queries, including single-keyword searches and multi-keyword 'AND' operations. A key feature is the provision of detailed results, which include the lines containing the search terms and a summary of how many times each keyword appears, providing a comprehensive solution for text-based information retrieval.

Objectives:

- To design and implement a solution for the given problem statement.
- To utilize modern tools and algorithms.
- To optimize performance and code quality.

5. Project Description & Methodology :

The development of the Text Search Indexer was approached in a sequential and modular manner. Initially, input texts were subjected to a rigorous tokenization and normalization process, converting text into a standardized format by lowercasing and removing non-alphanumeric characters to ensure consistent matching. Following normalization, a word-to-line index was constructed, mapping each unique word to a list of lines in which it appears. Search functionalities were then implemented, starting with single-keyword searches that leverage the index for direct lookup. Subsequently, a multi-keyword AND search was developed, requiring keywords to be present on the same line for a match. Finally, the system was enhanced to return search results augmented with occurrence count summaries for each keyword within the matched lines, providing a comprehensive view of search relevance.

6. Tools & Technologies Used :

Programming Language: JavaScript
Libraries/Frameworks: Standard JavaScript features (String manipulation, Array methods, Object literals)

Concepts: Text processing, Tokenization, Normalization, Indexing, Data Structures (Hash Maps/Objects, Arrays), Search Algorithms, Information Retrieval.

7. Implementation & Results :

Problem 1: Tokenize and Normalize Text

Description: The problem requires implementing a robust text processing pipeline that can handle diverse text inputs. This involves breaking down raw text into meaningful units (tokens) and transforming these tokens into a standardized format. Normalization typically includes converting text to lowercase and removing punctuation or special characters to ensure that variations in capitalization or spelling do not impede search accuracy. The goal is to prepare text data for effective indexing and searching by reducing its complexity and variability.

```
const fs = require('fs');

   let input = fs.readFileSync(0, 'utf-8');

   if (input.endsWith('\n')) {
   input = input.slice(0, -1); }

   const lines = input.split('\n');
   const results = [];

   for (const line of lines) {

       let result;

       if (line.trim() === '') {
   result = {"words": []};     }
   else {         const words =
   line             .toLowerCase()
               .replace(/[^a-z0-9\s]/g, '')
               .trim()
               .split(/\s+/)
   .filter(word => word !== '');

           result = {"words": words};
   }

       results.push(result);
   }

   console.log(JSON.stringify(results));
```

Problem 2: Build Word to Line Index

Description: This task focuses on creating an efficient data structure that maps words to their locations within the input texts. The objective is to build an index, specifically a word-to-line

mapping, where each unique word found in the text is associated with a list of line numbers on which it occurs. This index serves as the backbone for fast retrieval of documents or lines containing specific terms, enabling quick lookups during search operations.

```
if (input.endsWith('\n')) {
input = input.slice(0, -1); }

const lines = input.split('\n');

const results = [];

let index = {};

for (let i = 0; i < lines.length; i++) {
const line = lines[i];

    if (line.trim() === '') {
continue;     } else {
const words = line
.toLowerCase()
            .replace(/[^a-z0-9\s]/g, '')
.split(/\s+/);

        words.forEach(word => {
if (!index[word]) {
index[word] = [];             }

            if (!index[word].includes(i)) {
index[word].push(i);             }
        });
    }
}

results.push(index);

console.log(JSON.stringify(results));
```

Problem 3: Search for a Single Keyword

Description: The challenge is to develop a search functionality that can quickly locate all occurrences of a single user-specified keyword within the indexed text data. This involves querying the pre-built word-to-line index to retrieve all line numbers associated with the given keyword. The search must be efficient and accurate, returning the relevant lines where the keyword is present.

```
const results = [];

const index = {"hello":[0,1],"world":[0,1]};
```

```
const keyword = lines[0].trim().toLowerCase();

let found_lines = index[keyword] || [];

results.push(found_lines);

console.log(JSON.stringify(results));
```

Problem 4: Support Multi-Keyword AND Search

Description: This problem extends the search capability to handle queries involving multiple keywords, specifically implementing an 'AND' logic. For a multi-keyword AND search, the system must identify lines that contain *all* specified keywords. This requires efficiently intersecting the sets of line numbers associated with each keyword in the index. The solution must ensure that only lines satisfying all keyword conditions are returned as results.

```
const fs = require('fs');

let input = fs.readFileSync(0, 'utf-8');

if (input.endsWith('\n')) {
input = input.slice(0, -1); }

const lines = input.split('\n');
const results = [];

const index = {"hello":[0,1],"world":[0,1], "test": [1]};

const keywords = lines[0]
.toLowerCase()
    .trim()
.split(/\s+/);

let found_lines = [];

if (keywords.length > 0) {     found_lines
= index[keywords[0]] || [];

    for (let i = 1; i < keywords.length; i++) {
const linesForKeyword = index[keywords[i]] || [];

        found_lines = found_lines.filter(line =>
linesForKeyword.includes(line)         );
    }
}

results.push(found_lines);

console.log(JSON.stringify(results))
```

Problem 5: Return Results with Occurrence Count Summary

Description: The final requirement is to enhance the search results by providing a summary of how many times each queried keyword appears on the lines that match the search criteria. Beyond simply listing the matching lines, the system should offer a quantitative measure of keyword frequency within the context of the search results, thereby adding significant value to the returned information.

```
const results = [];

const index = {"hello":[0,1],"world":[0,1]};

const keywords = lines[0].toLowerCase().trim().split(/\s+/);

let counts = {};

for (const keyword of keywords) {     const
matchingLines = index[keyword] || [];

    counts[keyword] = matchingLines.length;
}

results.push(counts);

console.log(JSON.stringify(results));
```

8. Learning Outcomes

- Gained practical experience in implementing text processing techniques such as tokenizationand normalization.
- Developed a strong understanding of inverted indexing and its application in efficient textsearch.
- Learned to design and implement search algorithms for both single and multi-keyword queries.
- Enhanced skills in data structure manipulation, particularly using JavaScript objects and arraysfor indexing.
- Improved problem-solving abilities in handling complex string operations and logical searchconditions.

9. Challenges Faced & Solutions

Challenge: Ensuring case-insensitivity and handling of punctuation during tokenization and normalization.

Solution: Implemented a multi-step normalization process that converts text to lowercase and uses regular expressions to remove unwanted characters, ensuring consistent word matching. Challenge: Efficiently performing multi-keyword AND searches across potentially large datasets.

```
const fs = require('fs');

let input = fs.readFileSync(0, 'utf-8');

if (input.endsWith('\n')) {
input = input.slice(0, -1); }

const lines = input.split('\n');
```

Solution: Utilized set intersection operations on the lists of line numbers retrieved from the index for each keyword, allowing for rapid identification of lines containing all specified terms.
Challenge: Providing accurate occurrence counts for keywords within the context of search results.
Solution: After identifying matching lines, re-processed the text of those specific lines to count the occurrences of each keyword, rather than relying solely on the index which tracks presence rather than count.

10. Conclusion

The Text Search Indexer project successfully demonstrates a functional and efficient approach to text processing and information retrieval. By systematically addressing challenges in text normalization, indexing, and search, the system provides a robust tool for querying textual data. The implementation of single and multi-keyword searches, coupled with occurrence count summaries, highlights the practical application of core data structure and algorithm principles in solving real-world problems related to information access.

11. Future Scope

* Implement OR and NOT search operators for more complex query capabilities.
* Integrate fuzzy searching to handle misspellings and variations in keywords.
* Extend the indexer to support searching across multiple documents or files concurrently.

12. References

* JavaScript MDN Web Docs: String Methods - https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String
* JavaScript MDN Web Docs: Array Methods - https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array
* GeeksforGeeks: Inverted Index - https://www.geeksforgeeks.org/inverted-index/

13. Annexure

• Internship Completion Certificate
• Offer Letter